LangGraph Lab Report — Building AI Agents with LangGraph

Merouane Mouadili

DSAI2S Research Team, C3S Laboratory, FST of Tangier, Abdelmalek Essaâdi University, Tetouan, Morocco

1. Introduction

This lab moved step by step from a simple LLM call to a full agent that classifies questions, calls tools, remembers the conversation and handles errors. The final result is an Intelligent University Assistant built with LangGraph.

Item	What we used
Environment	Google Colab, Python 3.13
Libraries	LangGraph 1.2.19, LangChain 1.4.7, langchain-community, langchain-groq
LLM provider	Groq API (free), key created at console.groq.com/keys
Model	openai
API key handling	Entered with getpass (hidden input), never written in the notebook
Graph display	Mermaid.js rendered locally in the notebook
Extra tools	SQLite (fictitious PhD students database), Wikipedia API

The lab was first planned with OpenAI (gpt-4o-mini), but the OpenAI account had no credit (error 429, insufficient_quota). We switched to Groq. The first Groq model tried (llama-3.3-70b-versatile) was no longer available (error 404), so we listed the available models and chose openai/gpt-oss-120b for its good tool-calling support. Only the LLM definition changed; all LangGraph code stayed the same.

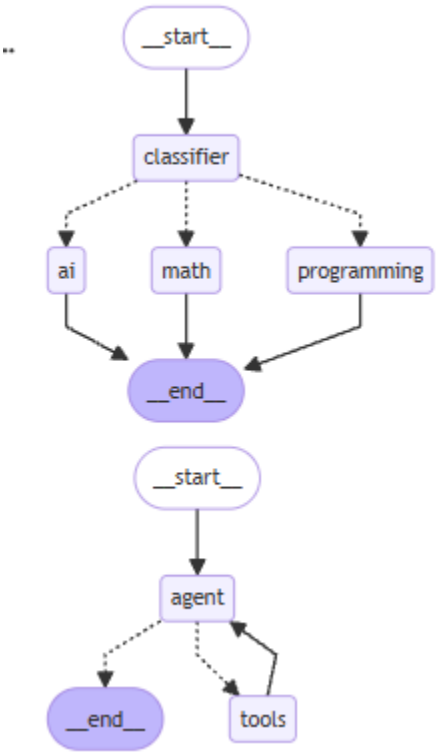
2. What we built

Each part added one capability to the previous one, and all tests gave the expected result.

Part	What we built	Result
II. First workflow	START → process →END with a fixed reply	State passed correctly between nodes
III–IV. LLM node	A chatbot node calling the LLM	Correct 3-sentence answer on AI
IV. Two-stage agent	analyzer →answer	More structured answer on reinforcement learning

V–VI. Conditional routing	classifier →math / programming / ai	3/3 questions routed correctly
VII. Tools	Unsafe eval calculator, safe ast calculator, date/time tool	110 computed; malicious input rejected
VIII. Tool-using agent	Agent → tools loop with tools_condition	Calculator called for 125 × 38 = 4,750; no tool for the capital of Morocco
IX. Memory	add_messages + MemorySaver + thread_id	Name remembered in the same thread, unknown in another thread
Visualization	get_graph() rendered with Mermaid	Both graphs displayed after a fix (section 5)
Evaluation	6 test questions on the router	Routing accuracy 6/6 = 100%
X. Capstone	Intelligent University Assistant	All categories, tools, memory and error cases work

The graphs of the router (Part VI) and of the tool-using agent (Part VIII), generated in the notebook:



The router branches to one of three nodes; the tool agent loops between agent and tools until no tool is needed.

3. Synthesis of the conceptual answers

The full answers to each conceptual question are written in the "Your answer" cells of the notebook (langgraph_lab_Statement.ipynb), together with an observation after each exercise. This section summarizes them.

In short: the LLM thinks, LangChain provides the building blocks, and LangGraph organizes them into a workflow that can decide, loop and remember.

Roles of LangChain, LangGraph and the LLM

- **LLM**: the reasoning engine. It understands the request and decides the next action, but alone it cannot act on the outside world or remember past conversations.
- **LangChain**: a toolbox of building blocks (prompts, model connectors, tools, parsers) that can be linked into a chain with a fixed, one-way path.
- **LangGraph**: builds agents as graphs, with conditions, loops and memory, so the model can choose what to do next.

Graph concepts

- **Node**: one step of the workflow, a function that reads the state and returns the values to update.
- **Edge**: a link that sets which node runs next. A conditional edge chooses the next node at run time from the state.
- **START / END**: the entry and exit points of the graph; they do no work themselves.
- **State**: the shared memory of the graph. Nodes cooperate through it without calling each other directly.

Chain vs agent

A chain always follows the same path. An agent decides its path: it can call a tool, read the result, and loop until it can answer. In our tests, the agent called the calculator for 125 × 38 but answered the capital of Morocco directly.

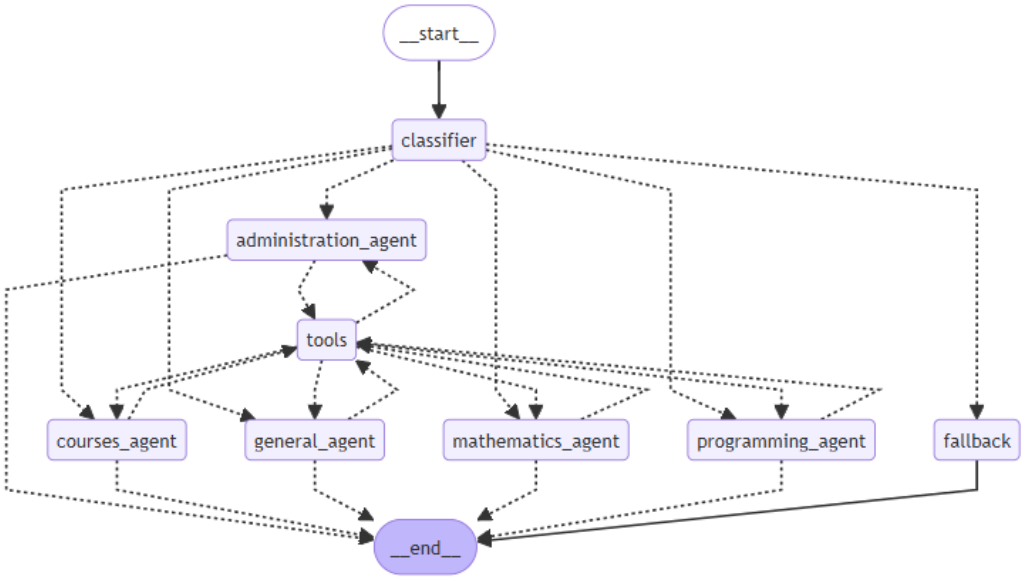
Router evaluation

The router made no error on the 6 test questions. Routing can be made more robust with structured output (a Literal type with with_structured_output), validation of the label with a fallback branch, few-shot examples, clear category definitions, and a larger test set including ambiguous questions.

4. Capstone: Intelligent University Assistant

The assistant classifies each question into one of five categories, sends it to a specialized agent, and lets that agent call shared tools before answering.

The graph below was generated in the notebook with `show_graph_local(assistant)` (LangGraph `get_graph()` rendered with Mermaid.js).



Dashed arrows are conditional edges. After a tool runs, control returns to the agent that asked for it, using the category stored in the state.

Key design choices:

- One agent per category, created by a factory function, each with its own system prompt.
- One shared tools node with handle_tool_errors=True, so a bad tool call becomes an error message instead of a crash.
- MemorySaver with a thread_id per conversation.
- Error handling: empty input goes to fallback without any LLM call, an unknown label defaults to GENERAL, API errors return a polite message.

Test query	**Category**	**Tool used**	**Result**
Topics of a machine learning course	COURSES	none	Full course outline
Python factorial function	PROGRAMMING	none	Correct, documented code
(15 + 27) × 3	MATHEMATICS	calculator	126
Enrollment certificate	ADMINISTRATION	none	Generic procedure, refers to the registrar
Today's date	GENERAL	current_datetime	Sunday, November 1, 2026
Empty input	EMPTY	none	Fallback message
My name is Ahmed, then What is my name?	GENERAL	none	Name remembered in the same thread

Sara El Amrani's supervisor	GENERAL	phd_database	Prof. Idrissi
Total publications of the LIST lab	GENERAL	phd_database	13
Who was Ibn Khaldun?	GENERAL	wikipedia	Answered from own knowledge, source flagged as unchecked

The PhD database (SQLite, parameterized queries) contains fictitious data only.

5. Problems encountered and solutions

All problems came from the environment or external services, not from the LangGraph logic, and each one was solved.

Problem	Cause	Solution
Error 429 with OpenAI	No credit on the account	Switched to the free Groq API
Error 404 with Groq	llama-3.3-70b-versatile no longer available	Listed available models, chose openai/gpt-oss-120b
API key not asked again	Old key kept in os.environ	Removed old keys before entering the new one
Graph PNG failed (error 400)	The tools node label contains a list that Mermaid cannot parse	Removed the list with a regular expression and rendered with Mermaid.js in the notebook
Unsafe calculator	eval() can run any Python code	Safe calculator based on ast, arithmetic only
Wikipedia tool called 12 times	API errors (JSONDecodeError); the agent repeated the same call	Tool rewritten with 2 retries max and a message telling the agent to stop and answer from its own knowledge; now 1 call only

6. Conclusion

The lab objectives were met: we built, visualized, tested and evaluated agents that route questions, use tools, keep memory and recover from errors. The main lesson is that an agent's behavior depends as much on its prompts and error handling as on the graph itself.